

Autodesk® Scaleform®

Scaleform 3Di

Scaleform 3.2

3D

Mustafa Thamer

2.03

2012 5 9

Copyright Notice

Autodesk® Scaleform® 4.2

© 2012 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Homestyler, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, Beast (design/logo) Built with ObjectARX (design/logo), Burn, Buzzsaw, CAiCE, CFdesign, Civil 3D, Cleaner, Mia, toane1

Autodesk Scaleform

Scaleform 3Di

Scaleform Corporation

6305 Ivy Lane, Suite 310

Greenbelt, MD 20770, USA

www.scaleform.com

info@scaleform.com

(301) 446-3200

(301) 446-3199

1		1
2	Flash 10	2
2.1		2
2.2		2
3	3D Flash	3
3.1		3
3.2	3Di.....	3
4	3Di API	4
4.1	ActionScript 2 API	4
4.2	C++ 3Di.....	7
4.2.1	Matrix4x4	7
4.2.2	Direct Access	7
4.2.3	Render::TreeNode	7
4.2.4	: Direct Access API	8
5		9
5.1	AS3	11
6	Stereoscopic 3D	12
6.1	NVIDIA 3D Vision.....	12
6.1.1	3D Vision	12
6.1.2		12
6.1.3		12
6.2	Scaleform Stereo API	14
6.2.1	Scaleform Stereo	14
7		16

1

Scaleform® 3Di™

UI

3D Flash

Scaleform 3.2 ,Scaleform

ActionScript

,

3D

Scaleform 3.2 ActionScript 2.0

,

3D AS

AS 3.0

(_z _zscale _xrotation _yrotation _matrix3d _perspfov),

HUD

UI

3D

3D

c + +

3D

Scaleform

API

API

3D GfX::Value

,

3D

3D Flash

,

3 3D Flash

Scaleform 3.2, 3D Flash

3.1

Flash , 3D Flash
Scaleform 3.2 , 3D
“(Render to Texture) 3Di Flash
3D , 3D , 3D
Z() / (UI)
, ,
Flash (Scaleform) ,
, Flash ,
, “SWF ”(SWF to Texture) , Scaleform SDK

3.2 3Di

(Scaleform 3.2) 3Di --- ActionScript Direct Access API (C++)
,3Di ,
– ,
Scaleform Scaleform
,3Di , Scaleform , UI,
, UI ,
UI 3D , UI
, UI ,

4 3Di API

4.1 ActionScript 2 API

3Di,Flash

,

, 3D , 3D (x y z

3d),

3d

“(Null)(, ActionScript ,foo._matrix3d = null),

2D

z

0

3D

,3D

3

(x y z)

,

Z

2D

,3D

,

_zscale ActionScript

Scaleform

3D

Flash 10

,

+X

,+Y

,

+Z

(

)

+Z

,FOV

55

3D

Flash

-
-
-

3Di

ActionScript

- **_z** - Z (), 0
- **_zscale** - Z , 100
- **_xrotation** - X() , 0
- **_yrotation** - Y() , 0
- **_matrix3d** - 16 (4 x 4) 3D
NULL, 3D , 2D
- **perspfov** - (FOV) > 0 < 180

-1

FOV

55

ActionScript

gfxExtensions

True()

```
,  
    _xrotation _yrotation 3D  
    ,  
    3Di  
    ,
```

1

Y 45

```
_global.gfxExtensions = true;  
mc1._yrotation = 45;
```

2

X Z

```
_global.gfxExtensions = true;  
sq1._zscale = 500;
```

```
function rotateAboutXAxis(mc:MovieClip):Void  
{  
    mc._xrotation = mc._xrotation + 1;  
    if (mc._xrotation > 360)  
    {  
        mc._xrotation = 0;  
    }  
}
```

```
onEnterFrame = function()  
{  
    rotateAboutXAxis(sq1);  
}
```

3

3D 3D

```
function PixelsToTwips(iPixels):Number
```

```

{
    return iPixels*20;
}

function TranslationMatrix(tX, tY, tZ):Array
{
    var matrixC:Array = [ 1, 0, 0, 0,
                          0, 1, 0, 0,
                          0, 0, 1, 0,
                          tX,tY,tZ,1];
    return matrixC;
}

this._matrix3d = TranslationMatrix(PixelsToTwips(100),PixelsToTwips(100),0);

```

4.2 C++ 3Di

4.2.1 Matrix4x4

Matrix4x4 4x4 3D
() , Render_Matrix4x4.h

4.2.2 Direct Access

Direct Access (Gfx::Value) 3Di API 3D
3Di ActionScript

, Gfx_Player.h [Gfx::Value::DisplayInfo](#)

```
void SetZ(Double z)
void SetXRotation(Double degrees)
void SetYRotation(Double degrees)
void SetZScale(Double zscale)
void SetFOV(Double fov)
void SetProjectionMatrix3D(const Matrix4F *pmat)
void SetViewMatrix3D(const Matrix3F *pmat)
bool SetMatrix3D(const Render::Matrix3F& mat)
```

```
Double GetZ() const
Double GetXRotation() const
Double GetYRotation() const
Double GetZScale() const
Double GetFOV() const
const Matrix4F* GetProjectionMatrix3D() const
const Matrix3F* GetViewMatrix3D() const
bool GetMatrix3D(Render::Matrix3F* pmat) const
```

4.2.3 Render::TreeNode

Render::TreeNode , 3D “ ”(View) “
”(Perspective) , ,

Gfx_Player.h

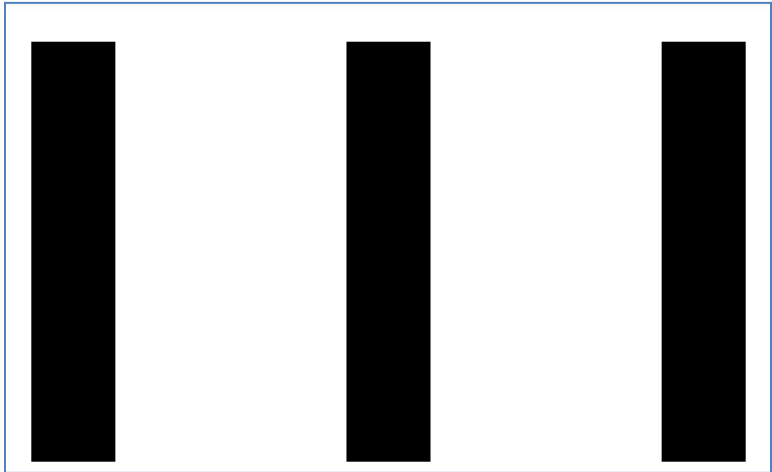
```
void    SetProjectionMatrix3D(const Matrix4F& m)
void    SetViewMatrix3D(const Matrix3F& m);
```

4.2.4 : Direct Access API

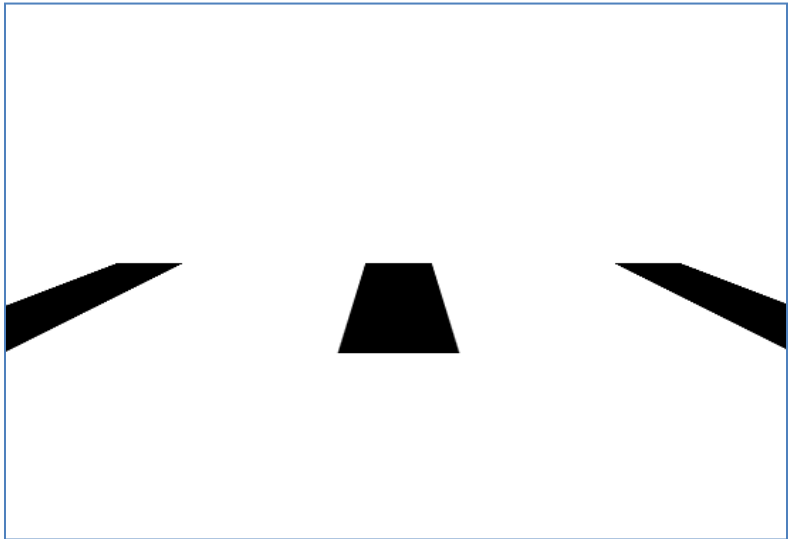
```
Ptr<GFx::Movie> pMovie = ...;
GFx::Value tmpVal;
bool bOK = pMovie->GetVariable(&tmpVal, "_root.Window");
if (bOK)
{
    GFx::Value::DisplayInfo dinfo;
    bOK = tmpVal.GetDisplayInfo(&dinfo);
    if (bOK)
    {
        // set perspectiveFOV to 150 degrees
        Double oldPersp = dinfo.GetFOV();
        dinfo.SetFOV(150);
        tmpVal.SetDisplayInfo(dinfo);
    }
}
```

Direct Access API (FOV) FOV 0, FOV

, z , FOV , 179 ,
0 180, 1, ,



2 Flash



3 -80 55 X

FOV (5)	.	,
。 ActionScript	property _perspfov	-1 C++
Direct Access API	:	

// 1. Perspective FOV -1

```

GFx::Value tmpVal;
bool bOK = pMovie->GetVariable(&tmpVal, "root"); //"_root" for AS2
if (bOK)
{
    GFx::Value::DisplayInfo dinfo;
    bOK = tmpVal.GetDisplayInfo(&dinfo);
    if (bOK)
    {
        dinfo.SetPerspFOV(-1);
        tmpVal.SetDisplayInfo(dinfo);
    }
}

```

// 2.

```

void MakeOrthogProj(const RectF &visFrameRectInTwips, Matrix4F *matPersp)
{
    const float nearZ = 1;
    const float farZ = 100000;
    float DisplayHeight = fabsf(visFrameRectInTwips.Height());
    float DisplayWidth = fabsf(visFrameRectInTwips.Width());
    matPersp->OrthoRH(DisplayWidth, DisplayHeight, nearZ, farZ);
}
GFx::Value tmpVal;
bool bOK = pMovie->GetVariable(&tmpVal, "root.mc");
if (bOK)
{
    GFx::Value::DisplayInfo dinfo;
    bOK = tmpVal.GetDisplayInfo(&dinfo);
    if (bOK)
    {
        Matrix4F perspMat;
        MakeOrthogProj(PixelsToTwips (pMovie->GetVisibleFrameRect()), &perspMat);
        dinfo.SetProjectionMatrix3D(&perspMat);
        tmpVal.SetDisplayInfo(dinfo);
    }
}

```

(Projection)

```

Ptr<Movie> pMovie = ...
// compute ortho matrix
Render::Matrix4F ortho;
const RectF &visFrameRectInTwips = PixelsToTwips(pMovie->GetVisibleFrameRect());
const float nearZ = 1;
const float farZ = 100000;
ortho.OrthoRH(fabs(visFrameRectInTwips.Width()),  fabs(visFrameRectInTwips.Height()),
              nearZ, farZ);
pMovie->SetProjectionMatrix3D(ortho);

```

5.1 AS3

ActionScript 3 , PerspectiveProjection()

Example:

```

import flash.display.Sprite;

var par:Sprite = new Sprite();
par.graphics.beginFill(0xFFCC00);
par.graphics.drawRect(0, 0, 100, 100);
par.x = 100;
par.y = 100;
par.rotationX = 20;

addChild(par);

var chRed:Sprite = new Sprite();
chRed.graphics.beginFill(0xFF0000);
chRed.graphics.drawRect(0, 0, 100, 100);
chRed.x = 50;
chRed.y = 50;
chRed.z = 0;

par.addChild(chRed);

var pp3:PerspectiveProjection=new PerspectiveProjection();
pp3.fieldOfView=120;
par.transform.perspectiveProjection = pp3;

```

6 Stereoscopic 3D

Scaleform 3Di UI, 3D
, , , ,
,
Scaleform 3Di 3D , ,PC NVIDIA 3D Vision Kit NVIDIA
, , , ()
, Display , ,
(Parallax Shift) Scaleform 4 PS3
TinyPlayerStereo(Scaleform):
Apps \Samples \GFxPlayerTiny \GFxPlayerTinyPS3GcmStereo.cpp

6.1 NVIDIA 3D Vision

6.1.1 3D Vision

NVIDIA 3D Vision Kit 3Di UI, NVIDIA
3D 3D (Stereoscopic
3D Test)

6.1.2

Scaleform Player 3D Vision
Direct3D Scaleform Player, , Scaleform Player
, command line argument -f NVIDIA ,
3D 3D , (CTRL + T)

6.1.3

, ,
, 3D Vision (Convergence)
,

, 3D Vision , IR ,
 (CTRL + F3) /(CTRL + F4) , , ,
 , Scaleform Player ,
 (CTRL + F5) (CTRL + F6), , ,
 , “Stereoscopic 3D”, “Set up stereoscopic 3D”, “Set
 Keyboard Shortcuts” NVIDIA “ ”(Enable the advanced in-game
 settings) , (CTRL + F5) (CTRL + F6) , (CTRL + F7)
 , , , , ,
 + F5), (CTRL + F5) (CTRL + F6), (CTRL
) 0 --- ,
 , (CTRL + F7) ,
 NVIDIA API, 3D ,
 3D ,

API

<http://developer.nvidia.com/object/nvapi.html>.

http://developer.download.nvidia.com/presentations/2009/SIGGRAPH/3DVision_Develop_Design_Play_in_3D_Stereo.pdf.

6.2 Scaleform Stereo API

3D ,Scaleform 4.1 Render::HAL API, 3D
HDMI 1.4
Scaleform 6.2.1

Scaleform 4

Render::Viewport::SetStereoViewport

SF4 (Viewport Stereo)

```
// ; ,  
  
View_Stereo_SplitV = 0x40,  
View_Stereo_SplitH = 0x80,  
View_Stereo_AnySplit = 0xc0,  
  
void SetStereoViewport(unsigned display);
```

6.2.1 Scaleform Stereo

```
Render::StereoParams s3DInfo;  
s3DInfo.DisplayDiagInches = 46; // 46 inch TV  
s3DInfo.DisplayAspectRatio = 16.f / 9.f;  
s3DInfo.Distortion = .75; // 0 to 1  
s3DInfo.EyeSeparationCm = 6.4; // this will default 6.4cm  
pRenderHAL->SetStereoParams(s3DInfo);
```

, Display , Display
Scaleform :

```
pMovie->Advance(delta);
```

```
pRenderer->GetHAL()->SetStereoDisplay(Render::StereoLeft);  
pMovie->Display();  
  
pRenderer->GetHAL()->SetStereoDisplay(Render::StereoRight);  
pMovie->Display();
```

7

3Di

Scaleform SDK

3D Flash

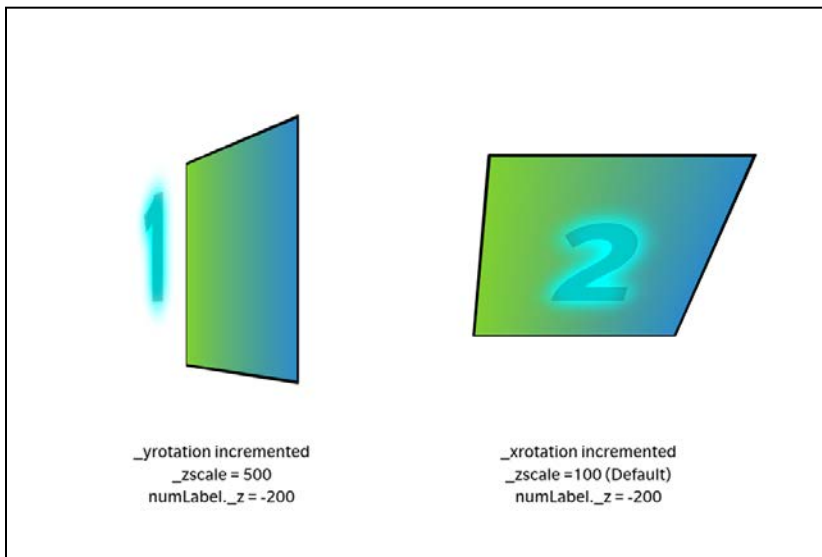
(SWF FLA)

,
AS3\Samples

C:\Program Files (x86)\Scaleform\GFx SDK 4.2\Bin\Data\AS2 or



4 3DGenerator



5 3D



6 3D